CUDA 编程模型与 GPU 内存层级：从 thread 到 HBM

> **原始问题：**继续深入讲解这部分：前置：CUDA 编程模型与 GPU 内存层级 —— 理解 kernel/thread/block、寄存器与 HBM，是理解上层一切性能优化的基础。

提问：2026/07/27 17:18　更新：2026/07/27 17:18　重要度：5/5

CUDA 编程模型与 GPU 内存层级：从 thread 到 HBM

一句话结论

CUDA 性能优化可以压缩成两条主线：

1. **执行层级决定“谁做哪份工作”**：CPU 发起 kernel；一个 kernel 形成 grid；grid 由多个 block 组成；block 由多个 thread 组成；硬件真正以 32 个 thread 构成的 warp 为调度单位，把 block 分配到 SM 上执行。
2. **内存层级决定“数据搬运有多贵”**：register 和 shared memory 在片上、快但容量小；L2 是全 GPU 共享缓存；global memory 物理上通常是 HBM，容量大但访问延迟高。大量 AI kernel 的优化本质是减少 HBM 往返并提高片上复用。

必须同时记住一个反直觉：**片上资源并非用得越多越好**。每线程寄存器和每 block shared memory 用量过大，会减少一个 SM 上能同时驻留的 block/warp，降低隐藏延迟的能力；强行压寄存器又可能导致 register spill，把数据落到慢得多的 local memory。

一、系统位置：为什么学上层框架仍必须懂 CUDA

在 PyTorch、Triton、Megatron 下面，最终都落到 CUDA 执行模型：

```
Megatron：决定模型如何分布到多卡
    ↓
PyTorch：组织 Tensor、Autograd 和算子
```

```
    ↓
Triton / cuBLAS / cuDNN / 自定义 CUDA kernel：决定单个算子如何实现
    ↓
CUDA：kernel launch、线程层级、内存、stream、同步
    ↓
GPU：SM、warp scheduler、register file、shared memory、L2、HBM
```

理解 CUDA 后，你才能解释这些上层现象：

上层现象	CUDA 层根因
`torch.compile` 融合算子后更快	减少 kernel launch 和中间 Tensor 的 HBM 读写
FlashAttention 比普通 Attention 省显存、更快	用 tiling 在 shared memory/register 中复用，避免完整 attention matrix 往返 HBM
Triton 的 `BLOCK_SIZE` 影响性能	决定一个 program/block 的工作粒度、访问合并、寄存器压力和 occupancy
NCCL 与计算 overlap 不充分	通信 kernel、计算 kernel、HBM 带宽和 SM 可能互相争用
同一个模型换 shape 后性能突变	grid/block 数量、tile 尺寸、边界 mask、数据复用率和 kernel 选择发生变化

二、统一例子：用 SAXPY 串起整篇

全文先使用一个统一、可手算的例子：

```
z[i] = a × x[i] + y[i]
```

参数：

```
N = 1,048,576 = 2²⁰ 个元素
数据类型 = float32 = 4 Byte/元素
blockDim.x = 256 threads/block
gridDim.x = ceil(N ÷ 256) = 4096 blocks
warp size = 32 threads
每个 block = 256 ÷ 32 = 8 warps
```

每个 thread 负责一个元素：

```
global_idx = blockIdx.x × blockDim.x + threadIdx.x
```

总逻辑线程数：

```
4096 blocks × 256 threads/block = 1,048,576 threads
```

这恰好覆盖全部元素。真实工程通常仍保留 `if (i < N)`，这样 `N` 不能整除 block size 时也安全。

每个元素的有用工作量：

```
读取 x[i]：4 Byte
读取 y[i]：4 Byte
写入 z[i]：4 Byte
合计 HBM 有用流量：12 Byte/元素
计算：1 次乘法 + 1 次加法 ≈ 2 FLOP/元素
```

整个 kernel：

```
有用数据量 = 1,048,576 × 12 Byte
           = 12,582,912 Byte
           = 12 MiB

计算量 ≈ 1,048,576 × 2 FLOP
       = 2,097,152 FLOP
       ≈ 2.10 MFLOP
```

算术强度：

```
AI = FLOP ÷ HBM Byte
   ≈ 2 ÷ 12
   ≈ 0.167 FLOP/Byte
```

这是非常低的算术强度，因此 SAXPY 通常是 **memory-bound**：核心问题不是 GPU 会不会乘加，而是能否高效地把 x、y 从 HBM 搬进来，再把 z 写回去。

三、CUDA 执行模型：host、device 与 kernel

3.1 Host 与 Device

- **Host**：通常指 CPU 及其内存。负责分配资源、准备数据、发起拷贝和 kernel launch。
- **Device**：GPU 及其显存。负责执行 kernel。
- **Kernel**：在 GPU 上被大量线程并行执行的函数。

典型控制流：

```
CPU 分配 host/device 内存
→ 把输入复制到 device
→ CPU 发起 kernel launch
→ GPU 异步执行 kernel
→ 必要时同步
→ 把结果复制回 host
```

Kernel launch 对 CPU 通常是异步的：CPU 把任务放进 stream 后可以继续提交后续任务。这是计算/通信 overlap、数据预取和 CUDA Graph 的基础。

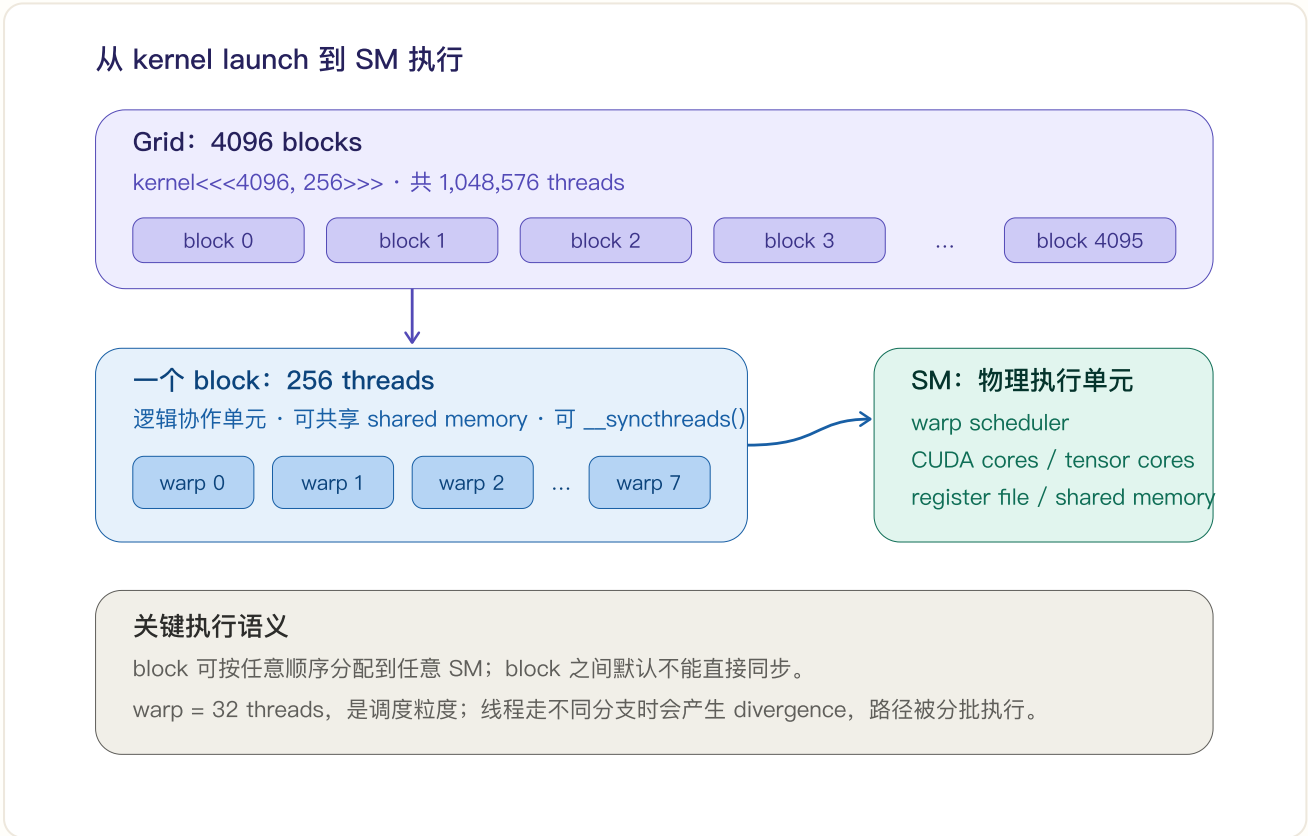
3.2 Grid、Block、Thread

层级关系：

```
一次 kernel launch
└── 一个 grid
    ├── block 0
    │   ├── thread 0
    │   ├── thread 1
    │   └── ...
    ├── block 1
    └── ...
```

三者职责：

层级	软件语义	硬件映射与约束
Thread	处理一个或多个数据元素	拥有私有寄存器和逻辑 local memory
Block	一组可协作的 thread	整个 block 驻留在一个 SM；可共享 shared memory，可做 block 内同步
Grid	本次 kernel 的全部 block	block 可按任意顺序分配到任意 SM；普通 kernel 中 block 间默认无直接同步



3.3 为什么 Block 必须彼此独立

CUDA 要让同一个 kernel 能在不同规模 GPU 上运行，block 的调度顺序不能成为正确性的前提：

- 某个 block 可能先执行，也可能后执行；
- 多个 block 可能同时驻留在不同 SM；
- grid 中 block 数可以远多于 SM 数，GPU 会分批执行；
- 普通 kernel 中不能假设“block 0 已经执行完，block 1 才开始”。

因此，若算法要求全 grid 同步，常见做法是：

1. 拆成两个 kernel；前一个 kernel 完成就是天然的全局边界；
2. 使用 cooperative groups 等特殊机制，但必须满足额外启动与驻留约束；
3. 用原子操作/计数器实现特定协议，但要非常谨慎，避免死锁和内存可见性错误。

四、Warp 与 SIMT：代码看似按 thread 写，硬件按 warp 执行

4.1 Warp 是什么

一个 warp 固定包含 32 个 thread，是 NVIDIA GPU 的基本调度粒度。以 `blockDim.x=256` 为例：

```
warp 0: threadIdx.x =   0..31
warp 1: threadIdx.x =  32..63
...
warp 7: threadIdx.x = 224..255
```

软件让每个 thread 有独立索引和寄存器状态，但 warp scheduler 每次选择一个 ready warp 发射指令。这种模型叫 **SIMT（Single Instruction, Multiple Threads）**。

4.2 Warp divergence

若同一 warp 内线程走不同分支：

```
if (threadIdx.x % 2 == 0) {
    path_a();
} else {
    path_b();
}
```

硬件不能让同一 warp 同一时刻真正执行两条不同指令流，通常需要用不同 active mask 分批执行：

```
先让偶数线程有效，执行 path_a
再让奇数线程有效，执行 path_b
```

于是两条路径都要付时间，未激活线程在对应阶段空转。关键不是“代码里有 if 就慢”，而是：

> **同一 warp 内是否发生分歧，以及各路径工作量是否显著。**

若分支条件按 warp 划分，例如 `warp_id < 4`，每个 warp 内所有线程走同一路径，通常不会产生同样的 divergence 代价。

4.3 Warp 如何隐藏延迟

HBM load 发出后往往不能立刻返回。GPU 不让当前 warp 阻塞整个 SM，而是切换到另一个 ready warp：

```
warp A 发出 HBM load → 等待
warp scheduler 改发 warp B 的算术指令
再发 warp C
等 A 数据就绪后继续 A
```

这就是 **latency hiding**。它要求 SM 上有足够多的 ready warps，所以 occupancy 有价值；但 ready warp 多不等于计算一定快，后面会讲为什么 100% occupancy 不是目标本身。

五、SM：Block 真正驻留和执行的地方

SM（Streaming Multiprocessor） 是 GPU 的主要计算单元。一个 SM 通常包含：

- warp scheduler / dispatch 单元；
- FP/INT 执行单元；
- Tensor Core；
- register file；
- shared memory / L1 相关片上资源；
- load/store、special function 等单元。

一个 block 一旦分配给某个 SM，它在生命周期内通常不会迁移到另一个 SM。一个 SM 可以同时驻留多个 block，但数量受到以下资源共同限制：

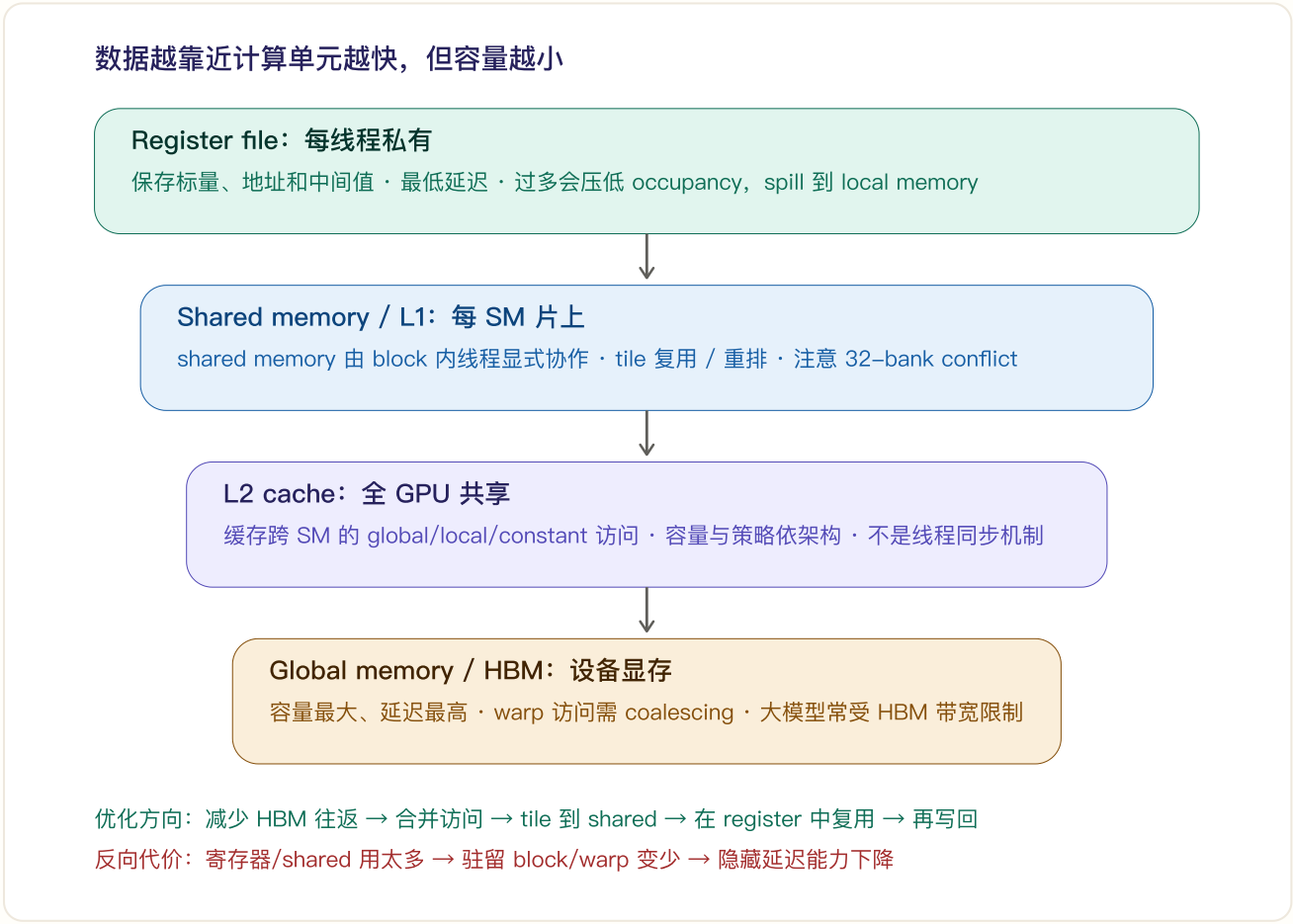
```
每 SM 最大 blocks
每 SM 最大 threads / warps
register file 总量
shared memory 总量
架构规定的其他限制
```

因此，配置 block size 时不是“线程越多越好”，而是要考虑：

```
并行度
× warp 对齐
× 寄存器压力
× shared memory 压力
× 数据访问模式
```

所有具体上限都依赖 GPU compute capability，不应把某一代 GPU 的数字当成 CUDA 永久规则。

六、GPU 内存层级：作用域、速度与管理方式



6.1 总览表

存储	典型作用域	物理位置/管理	典型用途	主要风险
Register	单 thread	每 SM 片上，编译器分配	地址、标量、中间累加值	用量过高降低 occupancy；spill
Local memory	逻辑上单 thread	通常位于设备显存，经 cache 访问	寄存器放不下的变量、动态索引数组、spill	名字含 local 但并不快
Shared memory	单 block	每 SM 片上，程序显式管理	tile、跨线程复用、数据重排	bank conflict；用量过高减少驻留 block
L1 cache	单 SM 附近	硬件管理，细节依架构	缓存部分 global/local 访问	命中率不稳定，不能代替正确同步
L2 cache	全 GPU	硬件管理	跨 SM 缓存 global/local 等流量	容量有限；不能假设数据总在 L2
Global memory / HBM	全 GPU	片外设备显存	模型参数、激活、KV、输入输出	高延迟；访问不合并会浪费带宽
Constant memory	全 grid 只读	设备内存 + 专用缓存	warp 内线程读取同一常量	访问地址高度分散时收益下降

6.2 Register：最快，但不是无限

源代码中的局部标量通常优先放寄存器：

```
float acc = 0.0f;
```

每个 thread 有自己的 `acc`。寄存器访问极快，也是 GEMM/Attention 把 tile 中间结果保存在片上的关键。

但寄存器消耗按 thread 乘开：

```
每 block 寄存器需求
≈ registers_per_thread × threads_per_block
```

如果每 thread 使用太多寄存器，一个 SM 能同时驻留的 block 就会减少。如果编译器无法分配足够寄存器，变量可能 spill 到 local memory；local memory 物理上通常在 HBM，代价远高于 register。

6.3 Shared memory：程序员管理的片上缓存

Shared memory 对同一个 block 的 thread 可见，典型模式：

```
所有 thread 合并读取 HBM 的一块 tile
→ 写入 shared memory
→ __syncthreads()
```

```
→ 每个 thread 多次复用 tile
→ 计算结果写回 HBM
```

这正是 tiled GEMM、卷积和 FlashAttention 的基础思想。

6.4 L1 / L2：硬件缓存，不等于共享变量

Cache 可以减少重复访问 HBM，但不能把“缓存可见”误解成“线程已经同步”。线程间正确通信仍需要：

- 正确的内存作用域；
- 原子操作或同步原语；
- 对应的 memory ordering / visibility 语义。

6.5 Global memory / HBM

CUDA 代码里的 global memory 是逻辑地址空间；在离散 GPU 上，其主要物理承载通常是 GDDR/HBM 设备显存。大模型中的参数、激活、梯度、优化器状态和 KV cache 大多长期驻留于这里。

HBM 有很高的吞吐带宽，但单次访问延迟仍高。GPU 依靠三种方式缓解：

1. 大量 warps 隐藏延迟；
2. warp 合并访问，提高每次事务的有效字节比例；
3. 用 register/shared/cache 提高数据复用，减少访问次数。

七、Global Memory Coalescing：相邻线程应访问相邻地址

7.1 SAXPY 的理想访问

一个 warp 的 32 个线程访问：

```
thread 0  → x[i+0]
thread 1  → x[i+1]
...
thread 31 → x[i+31]
```

float32 每元素 4 Byte，因此一个 warp 请求：

```
32 × 4 Byte = 128 Byte
```

对 compute capability 6.0 及之后的常见规则，可把它理解为覆盖若干个 32-Byte segment。若起始地址对齐，连续 128 Byte 理想情况下覆盖 4 个 segment：

```
4 segments × 32 Byte = 128 Byte
```

请求字节和实际搬运字节接近，效率高。

7.2 Stride 访问为什么浪费

若每个线程访问 `x[2 × i]`：

```
thread 0 → x[0]
thread 1 → x[2]
thread 2 → x[4]
...
```

相邻线程地址相差 8 Byte，但每次只使用其中 4 Byte。一个 warp 的 128 Byte 有用数据会散布在约 256 Byte 地址范围，理想效率约：

```
128 Byte useful ÷ 256 Byte transferred ≈ 50%
```

更大的 stride 可能让每个线程落到不同 segment，事务数急剧增加。

7.3 Requested 与 Actual Bandwidth

```
Load/Store Efficiency
≈ Requested bytes ÷ Actual transferred bytes
```

- Requested：程序真正用到的字节；
- Actual：内存系统为了完成请求实际搬运的字节。

两者差距大，通常意味着访问未合并、未对齐或读取了大量无用 cache line。

7.4 常用修复

- 让连续 thread 访问连续元素；
- 让最内层连续维映射到 `threadIdx.x`；
- 数据布局在 AoS 与 SoA 之间选择时，以 warp 访问模式为准；
- 对矩阵列访问/转置，先合并读入 shared memory，再在片上重排；

- block size 常选 32 的倍数，但最终仍需 profile。

八、Shared Memory Bank Conflict

Shared memory 被划分为多个 bank。对较新的常见 NVIDIA 架构，可用"32 个 bank、连续 32-bit word 映射到连续 bank"建立心智模型；具体模式仍应查对应架构文档。

对 float32 的简化映射：

```
bank_id = (address ÷ 4 Byte) mod 32
```

8.1 无冲突

warp 的 32 个线程访问 32 个不同 bank，可并行服务。

8.2 n-way conflict

多个线程访问同一个 bank 中的不同地址，访问需要拆分，吞吐可能近似降为原来的 `1/n`。注意：多个线程读取完全相同地址通常可广播，不按普通冲突处理。

8.3 经典转置例子

```
__shared__ float tile[32][32];
```

按列读取时，线程间步长是 32 个 float：

```
32 mod 32 = 0
```

不同线程可能都命中同一 bank，形成 32-way conflict。经典 padding：

```
__shared__ float tile[32][33];
```

此时：

```
33 mod 32 = 1
```

线程依次落到不同 bank，冲突消失。多出来的一列不是存业务数据，而是改变地址映射。

九、Occupancy：用于隐藏延迟，不是最终 KPI

Occupancy 定义：

```
Occupancy = active warps per SM ÷ maximum warps per SM
```

一个 block 的 warp 数：

```
warps_per_block = ceil(threads_per_block ÷ 32)
```

以 256 threads/block 为例：

```
warps_per_block = 256 ÷ 32 = 8 warps
```

教学近似下，能驻留的 block 数由最紧资源决定：

```
active_blocks_per_SM
≈ min(
    架构 block 上限,
    floor(SM thread 上限 ÷ threads_per_block),
    floor(SM register 总量 ÷ 每 block register 需求),
    floor(SM shared memory 总量 ÷ 每 block shared memory 需求)
  )
```

真实硬件会按特定分配粒度取整，准确结果应使用编译器报告、CUDA Occupancy API 或 Nsight Compute。

为什么 100% Occupancy 不保证最快

- memory-bound kernel 在已有足够 warps 隐藏延迟后，再提高 occupancy 可能无收益；
- 为提高 occupancy 强行减少寄存器，可能导致 spill，反而增加 HBM 流量；
- 减少 shared memory 可能失去数据复用，虽然 occupancy 上升但访问 HBM 更多；
- compute-bound kernel 可能更受 Tensor Core 利用率、指令依赖和 tile 形状影响。

正确目标是：

> 在不牺牲数据复用和制造 spill 的前提下，提供足够 ready warps 隐藏延迟。

十、Roofline：判断该优化计算还是优化内存

算术强度：

```
AI = FLOP ÷ Byte
单位：FLOP/Byte
```

Roofline 的简化上界：

```
可达到性能
≤ min(峰值计算吞吐，可达到内存带宽 × AI)
```

转折点：

```
AI_ridge = 峰值计算吞吐 ÷ 可达到内存带宽
```

- `AI < AI_ridge`：更可能 memory-bound；
- `AI > AI_ridge`：更可能 compute-bound。

SAXPY 的 `AI≈0.167 FLOP/Byte` 很低，因此优先优化：

- coalescing；
- 减少额外读写；
- 批量处理与融合；
- 避免不必要 dtype 转换；
- 检查真实 HBM throughput。

而大 GEMM 通过 tiling 让输入 tile 被反复使用，AI 随问题规模和 tile 复用提升，更可能进入 compute-bound 区域，此时重点变成 Tensor Core、矩阵 shape、数据类型、流水线与指令吞吐。

十一、有效带宽：如何量化 memory-bound kernel

有效带宽：

```
BW_effective = (Bytes_read + Bytes_written) ÷ kernel_time
```

单位检查：

```
Byte ÷ second = Byte/s
```

对本例：

```
有用数据量 = 12,582,912 Byte
```

若 CUDA Event 测得 kernel 时间为 `50 μs = 0.00005 s`：

```
BW_effective
= 12,582,912 Byte ÷ 0.00005 s
= 251,658,240,000 Byte/s
≈ 251.7 GB/s（十进制）
```

这个数字是“有用字节带宽”，不是实际 DRAM 搬运量。如果访问不合并，actual throughput 可能更高但有效带宽很低——说明总线在搬很多程序没用到的字节。

计时要注意：kernel launch 异步，不能只用普通 CPU 时钟包住 launch 而不做同步。优先使用 CUDA Event；教学测试也可在边界调用同步，但不要把大量同步留在生产热路径。

十二、同步：同步谁、保证什么

12.1 `__syncthreads()`

作用范围是一个 block：

- block 内所有参与线程到达 barrier 后才继续；
- 可用于 shared memory 的生产者/消费者阶段切换；
- 不能同步其他 block。

危险模式：只有部分线程进入 `__syncthreads()`，其他线程不进入，可能造成死锁或未定义行为。

12.2 Warp 级同步

Warp shuffle、vote、`__syncwarp()` 等用于 warp 内协作。现代 GPU 不应依赖“同 warp 天然锁步”来省略所有同步；使用 warp-level primitive 时仍要遵循其 mask 和内存可见性要求。

12.3 Kernel 边界

同一 stream 中，前一个 kernel 完成后后一个 kernel 才开始，因此拆 kernel 是常见的全 grid 同步方式。代价是额外 launch 和可能的中间数据写回 HBM。

12.4 Stream 与 Event

- 同一 stream 内按提交顺序执行；
- 不同 stream 的任务在无依赖时可以并发；
- Event 用来表达跨 stream 依赖；
- 不同 stream 能否真正重叠，还取决于资源是否冲突。

例如计算 kernel 和 NCCL kernel 虽在不同 stream，仍可能争抢 SM 或 HBM 带宽，所以时间线上有交叠不等于性能线性相加。

十三、最小 CUDA C++ 示例：SAXPY

```
#include <cuda_runtime.h>
#include <cstdio>

__global__ void saxpy(float a, const float* x, const float* y,
                      float* z, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        z[i] = a * x[i] + y[i];
    }
}

int main() {
    constexpr int N = 1 << 20;          // 1,048,576
    constexpr int threads = 256;
    const int blocks = (N + threads - 1) / threads;  // 4096
    const size_t bytes = N * sizeof(float);

    float *x, *y, *z;
    cudaMalloc(&x, bytes);
    cudaMalloc(&y, bytes);
    cudaMalloc(&z, bytes);

    saxpy<<<blocks, threads>>>(2.0f, x, y, z, N);
    cudaDeviceSynchronize();

    std::printf("blocks=%d threads/block=%d total_threads=%d\n",
                blocks, threads, blocks * threads);

    cudaFree(x);
    cudaFree(y);
    cudaFree(z);
}
```

这段代码展示的是线程映射，不是完整 benchmark：没有初始化输入、错误检查和 CUDA Event 计时。生产代码还应检查每个 CUDA API 和 kernel launch 的错误。

关键索引解释

```
blockIdx.x   当前 block 在 grid 中的编号
blockDim.x   每个 block 的 thread 数
threadIdx.x 当前 thread 在 block 内的编号
```

所以：

```
i = blockIdx.x × blockDim.x + threadIdx.x
```

把二维层级压成一维全局索引。

十四、PyTorch 视角：同一计算最终仍落到 CUDA Kernel

```
import torch

N = 1 << 20
torch.manual_seed(0)

x = torch.randn(N, device="cuda", dtype=torch.float32)
y = torch.randn(N, device="cuda", dtype=torch.float32)
a = 2.0

z = a * x + y
print(z.shape, z.dtype, z.device)
```

Eager 模式可能把乘法和加法分成多个 kernel，并产生中间 Tensor；`torch.compile`/Inductor 可能把它融合成一个 Triton/CUDA kernel。融合收益来自：

```
未融合：读 x → 写临时值 → 再读临时值和 y → 写 z
融合：读 x 和 y → register 中乘加 → 写 z
```

核心不是“少了一次 Python 调用”，而是**少了中间值的 HBM 往返和 kernel launch**。

十五、从 SAXPY 到矩阵乘：为什么 Shared Memory 能改变瓶颈

SAXPY 中每个输入只用一次，shared memory 很难创造复用；硬搬进 shared 反而多一道操作。

矩阵乘 `C = A × B` 不同。一个 A tile 和 B tile 会参与多次乘加：

```
合并从 HBM 读取 A/B tile
→ 放进 shared memory
→ block 内多个 thread 反复读取 tile
→ 每个 thread 在 register 中累加 C 的局部结果
→ 最终写回 HBM
```

这形成三层复用：

层级	保存内容	复用范围
HBM/L2	完整 A、B、C	全 GPU
Shared memory	当前 A/B tile	一个 block
Register	当前 thread 的累加器/fragment	单 thread

FlashAttention 也遵循相同总原则：通过 tile 和在线归一化把 Q/K/V 的局部块留在片上，避免把完整 `S×S` attention score matrix 写入 HBM。

十六、工程调优顺序：不要一上来就猜 block size

推荐顺序：

1. **确认正确性和 shape**：边界条件、对齐、dtype、索引是否正确。
2. **用 Profiler 定位热点**：先确认瓶颈确实在这个 kernel。
3. **计算 AI 和有效带宽**：判断 memory-bound 还是 compute-bound。
4. **检查 global access**：requested/actual throughput、coalescing、stride、对齐。
5. **检查片上复用**：是否可以用 register/shared 减少 HBM 流量。
6. **检查 shared bank conflict**：尤其是转置和二维 tile。
7. **检查资源压力**：register、shared memory、active blocks/warps、spill。
8. **检查 warp stall reason**：等待内存、执行依赖、同步还是指令供给。
9. **扫参数而不是猜参数**：block size、tile size、num warps、pipeline stages。

10. **回到端到端指标**：单 kernel 快不代表训练 step 一定快；还要看通信、CPU launch、同步和其他 kernel。

常用工具：

- Nsight Systems：看 CPU/GPU 时间线、stream、通信与计算 overlap；
- Nsight Compute：看单 kernel 的内存事务、occupancy、stall、吞吐；
- PyTorch Profiler：从框架算子关联到 CUDA kernel；
- 编译器报告：寄存器用量、spill、shared memory 使用量。

十七、常见误区

误区 1：Thread 是 GPU 的真实调度单位

不是。CUDA 暴露 thread 语义，但 NVIDIA GPU 以 warp 为基本调度单位。性能分析必须看 warp 的分支和访存模式。

误区 2：Local memory 在片上，所以很快

错误。Local 表示“线程私有的逻辑地址空间”，通常物理上位于设备显存，经 L1/L2 缓存访问；register spill 到 local memory 可能很贵。

误区 3：Shared memory 一定比 L1 快，所以都应该手动搬

错误。只有存在复用、重排或协作价值时才值得使用 shared。SAXPY 输入只用一次，额外搬运和同步可能使性能更差。

误区 4：Occupancy 越高越快

错误。Occupancy 是隐藏延迟的手段。若为提高 occupancy 导致寄存器 spill 或失去 shared tile 复用，性能会下降。

误区 5：Kernel launch 的 thread 数就是同时运行的物理线程数

错误。Grid 可以包含数百万逻辑 thread，但硬件只让受资源限制的一部分 block/warp 同时驻留，其他排队分批执行。

误区 6：有两个 stream 就一定能 overlap

错误。只有任务无数据依赖、设备支持并发、并且 SM/HBM/copy engine 等资源没有完全冲突时，才会出现有收益的真实 overlap。

十八、关键总结

把整篇压缩为一条因果链：

```
Kernel 把问题展开成大量逻辑 threads
→ threads 按 block 组织，block 被分配到 SM
→ 硬件以 warp 为调度单位执行
→ warp 遇到 HBM 延迟时切换到其他 ready warp
→ coalescing 决定一次内存事务有多少字节真正有用
→ shared/register 提高片上复用，减少 HBM 往返
→ 但 register/shared 用量过大会降低驻留 warp 数
→ 所以最终性能是“数据复用、访存效率、并行度、资源压力”的平衡
```

你应记住四个判断问题：

1. **映射**：每个 thread/block 到底处理哪块数据?
2. **访问**：同一个 warp 是否访问连续、对齐的地址?
3. **复用**：数据能否在 register/shared 中多用几次，少去 HBM?
4. **资源**：寄存器/shared 用量是否压低了 active warps，或造成 spill?

可选自测

1. 索引与 warp 题

`N=1,000,000`，`blockDim.x=256`：

- 需要多少个 block?
- 最后一个 block 有多少个有效 thread?
- 每个完整 block 有多少个 warp?

2. 带宽与算术强度题

对 float32 SAXPY `z=a*x+y`，若 `N=2²⁰`、kernel 时间 `40 µs`：

- 有用数据量是多少 MiB?
- 有效带宽是多少 GB/s（十进制）？
- 算术强度是多少 FLOP/Byte?
- 它更可能 memory-bound 还是 compute-bound?

3. 工程决策题

一个矩阵转置 kernel 的 Nsight Compute 显示：global load 基本合并，但 shared memory 有严重 32-way bank conflict；修改 padding 后 occupancy 从 75% 降到 62.5%。你是否应该保留 padding？还需要看哪些指标才能决定?

继续学习

1. 前置：GPU 微架构与 SM/Tensor Core —— 深入理解 warp 指令如何落到执行管线。
2. 同层对比：Triton 编程模型 —— 对比 CUDA 的 thread/block 与 Triton 的 program/block abstraction。
3. 下游应用：FlashAttention 的 IO-aware tiling —— 把 HBM、shared memory、register 和在线 softmax 连成真实大模型算子。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与边界

本文依据 NVIDIA CUDA Programming Guide 与 CUDA C++ Best Practices Guide 的稳定概念整理。warp 大小、常见 bank 模型和 coalescing 示例按当前 NVIDIA CUDA 文档说明；每个 SM 的最大线程数、寄存器总量、shared memory 容量、cache 组织和异步拷贝能力均随 compute capability 变化，生产调优必须以目标 GPU、CUDA 版本和 profiler 实测为准。